

Backend

Arquitectura Backend

El backend del sistema se diseña siguiendo una arquitectura en capas (Layered Architecture) bajo el patrón MVC, implementado en .NET 9 utilizando C#. Este enfoque organiza el sistema en componentes claramente separados según su responsabilidad, lo cual facilita la mantenibilidad, escalabilidad y claridad del diseño

La estructura propuesta se compone de las siguientes capas principales:

Controllers → Services → Repositories → Database

Cada capa interactúa únicamente con la inmediatamente inferior, evitando dependencias innecesarias y promoviendo un bajo acoplamiento entre componentes

Rol de cada capa

La capa de Controllers es el punto de entrada al sistema. Su responsabilidad principal es recibir las solicitudes HTTP provenientes del cliente (en este caso, la aplicación móvil), validar los datos básicos de entrada y delegar la ejecución de la lógica al Service correspondiente. Esta capa no contiene lógica de negocio, sino que actúa como intermediario entre el cliente y el sistema

La capa de Services encapsula la lógica de negocio de la aplicación. Aquí se procesan las reglas del sistema, como validaciones más complejas, decisiones de flujo y transformaciones de datos. Esta capa coordina las operaciones necesarias para cumplir con cada caso de uso, utilizando los Repositories para acceder a la información persistente

La capa de Repositories se encarga del acceso a datos. Su función es interactuar directamente con la base de datos, ejecutando operaciones CRUD (Create, Read, Update, Delete). Esta capa abstrae los detalles de la persistencia, permitiendo que el resto del sistema no dependa de la tecnología específica utilizada (en este caso, SQL Server)

La base de datos relacional (SQL Server) almacena la información del sistema de manera estructurada. En este contexto, contendría entidades como espacios de coworking, reservaciones y usuarios, permitiendo mantener la integridad y consistencia de los datos

Flujo de una petición

El flujo de una petición inicia cuando el cliente (la aplicación móvil) realiza una solicitud HTTP a un endpoint del backend, por ejemplo, para consultar los espacios disponibles o realizar una reservación

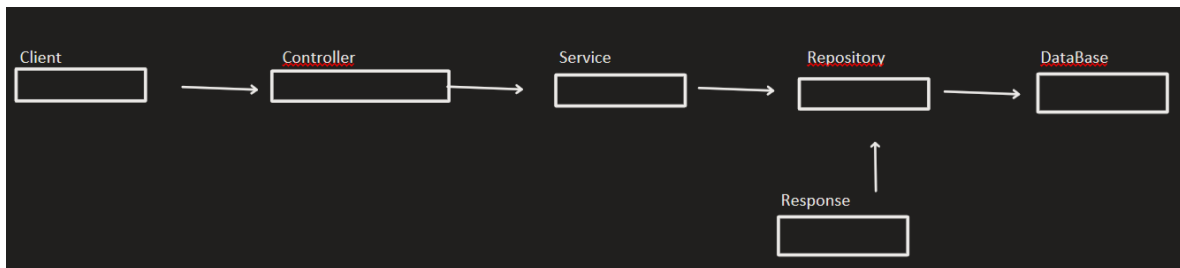
En primer lugar, la solicitud es recibida por un Controller, el cual interpreta el request y extrae los parámetros necesarios. Posteriormente, el Controller invoca al Service correspondiente, delegando la ejecución de la operación

El Service procesa la solicitud aplicando la lógica de negocio necesaria. Por ejemplo, podría validar la disponibilidad de un espacio antes de permitir una reservación. Para obtener o persistir datos, el Service utiliza uno o más Repositories

El Repository ejecuta las consultas necesarias contra la base de datos, ya sea para recuperar información o para almacenar nuevos registros. Una vez obtenidos los datos, estos son retornados al Service.

El Service procesa la respuesta, la adapta al formato requerido y la envía de vuelta al Controller, el cual finalmente construye la respuesta HTTP que será enviada al cliente

Este flujo puede representarse de forma simplificada como:



Justificación de decisiones

La elección de una arquitectura en capas responde a la necesidad de mantener una clara separación de responsabilidades dentro del sistema. Este enfoque permite que cada componente tenga un propósito específico, lo cual facilita la comprensión del código y reduce la complejidad al momento de realizar cambios

Además, esta arquitectura mejora la mantenibilidad, ya que modificaciones en una capa, como cambios en la base de datos, no afectan directamente a las demás

capas siempre que se respeten las interfaces definidas. Esto resulta especialmente valioso en sistemas que evolucionan con el tiempo

Otro aspecto importante es la escalabilidad. La separación en capas permite extender funcionalidades, como la incorporación de autenticación real, manejo de pagos o integración con servicios externos, sin necesidad de rediseñar completamente el sistema

Asimismo, el uso del patrón Repository permite desacoplar la lógica de negocio de los detalles de persistencia, facilitando pruebas unitarias y permitiendo cambiar la tecnología de base de datos en el futuro si fuera necesario

Finalmente, el uso de .NET 9 con arquitectura MVC se alinea con estándares ampliamente adoptados en la industria, proporcionando una base robusta, segura y eficiente para el desarrollo de APIs RESTful